

AiCoreMed

Sistema de Gestión para Consultorios Médicos
con IA Local, Automatizaciones y Multi-Tenant

Plan de producto multi-cliente + hoja de ruta Fase 1
Versión: 1.0 — 17 de mayo de 2026

1. Estado Actual del Producto

El sistema actual es una solución completa para UN consultorio médico con:

- Dashboard web (Next.js 14 + shadcn/ui + TypeScript)
- Automatizaciones n8n (7 workflows: WhatsApp, turnos, recordatorios, correo, resumen, recetas, backup)
- IA local con Ollama + Mistral (sin costos de OpenAI)
- Comunicación WhatsApp via Twilio
- PostgreSQL como base de datos
- Infraestructura Docker lista para Dokploy

Es un producto funcional de instancia única (single-tenant). El dashboard está 100% operativo en desarrollo con 8 secciones, seguridad completa (9 fases), 59 tests, y documentación exhaustiva.

2. Modelo de Negocio

Tres opciones para vender el producto:

Opción 1: Multi-tenant cloud (recomendada para Fase 1)

Todos los clientes comparten la misma infraestructura. Cada uno accede via subdominio (cliente.aicoremed.com). Los datos se aíslan por tenant_id con RLS en PostgreSQL.

- Un solo deploy, actualizaciones instantáneas
- Menor costo operativo por cliente
- Ideal para empezar a vender rápido

Opción 2: Single-tenant (instancias separadas)

Cada cliente tiene su propio stack completo via Dokploy. Script de provisioning que crea DB + dashboard + n8n + subdominio.

- Aislación total, compliance on-premise
- Mayor consumo de recursos
- Ideal para plan Enterprise u on-premise

Opción 3: Híbrida (recomendada a largo plazo)

Dashboard multi-tenant cloud para la mayoría de clientes + opción on-premise para Enterprise. Panel de administración centralizado que controla todos los clientes (creación, monitoreo, facturación, actualización).

3. Pricing Sugerido

Plan	Precio	Incluye
Starter	\$49/mes	1 consultorio, 500 pacientes, WhatsApp, IA básica
Professional	\$99/mes	1 consultorio, pacientes ilimitados, IA avanzada, reportes
Multi	\$199/mes	Hasta 3 consultorios, todo incluido
Enterprise	\$499/mes	Consultorios ilimitados, on-premise, SLA, soporte prioritario
On-premise	\$299/mes	Licencia para instalar en su propia VPS

Costos operativos mensuales estimados: VPS \$20-40, Twilio ~\$0.005/msg (variable), Dominio+SSL ~\$1/mes, Ollama gratis.

Margen estimado: sobre plan Professional (\$99/mes), el costo infraestructura es de ~\$30-50/mes compartido entre varios clientes. Con 10 clientes en Professional, el margen es de ~\$900-950/mes.

4. Arquitectura Multi-Tenant

4.1 Tabla tenants

```
CREATE TABLE tenants (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nombre VARCHAR(255) NOT NULL,  
  subdomain VARCHAR(100) UNIQUE NOT NULL,  
  logo_url TEXT,  
  colores JSONB,  
  config JSONB,  
  activo BOOLEAN DEFAULT true,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

4.2 Aislamiento con tenant_id

Cada tabla existente recibe una columna tenant_id UUID REFERENCES tenants(id). Se aplica Row-Level Security en PostgreSQL:

```
ALTER TABLE pacientes ENABLE ROW LEVEL SECURITY;  
CREATE POLICY tenant_isolation ON pacientes  
  USING (tenant_id = current_setting('app.tenant_id')::UUID);
```

4.3 Middleware de ruteo por subdominio

```
const host = request.headers.get('host') || '';  
const subdomain = host.split('.')[0];  
const tenant = await getTenantBySubdomain(subdomain);  
if (!tenant) return NextResponse.redirect('/404');  
request.headers.set('x-tenant-id', tenant.id);
```

4.4 n8n multi-tenant

Los workflows n8n reciben tenant_id como parámetro en cada ejecución. La query a base de datos filtra por ese ID. Las credenciales de Twilio se resuelven según el tenant.

4.5 White-label dinámico

Logo, colores, nombre del consultorio se leen de la tabla tenants y se aplican en runtime. No requiere rebuild del dashboard por cliente.

5. Hoja de Ruta — Fase 1

Duración estimada: ~15-17 días hábiles para MVP multi-tenant vendible.

Orden recomendado de implementación:

#	Tarea	Tiempo	Detalle
1	Tabla tenants + columna tenant_id	2 días	Migración 009 + actualizar todas las queries existentes
2	Middleware subdominio + sesión	1 día	Extraer tenant del host, setear en headers y sesión
3	Login multi-tenant	1 día	Cada tenant con sus propios usuarios, login por subdominio
4	RLS + filters en queries	1 día	PostgreSQL RLS + Drizzle/JSON filters por tenant_id
5	Panel admin de tenants	2 días	CRUD de clientes, activar/desactivar, ver uso
6	Script provisioning automático	1 día	Crear tenant, DB seed, subdominio, n8n credentials
7	White-label dinámico	1 día	Logo, colores, nombre desde DB, CSS variables en runtime
8	PWA (app instalable)	1 día	manifest.json, service worker, iconos
9	Landing page + registro	2 días	Página pública con pricing, formulario registro, onboarding
10	Pasarela de pago	2 días	Stripe/MercadoPago, webhooks, planes recurrente
11	Dashboard de uso por cliente	1 día	Stats: requests, almacenamiento, mensajes Twilio

Testing + ajustes

1 día

Tests multi-tenant, fixes, build final

6. Qué Podemos Empezar a Hacer Ahora Mismo

Sin esperar nada más. Con el código actual podemos arrancar hoy:

Paso 1: Migration 009 — Tabla tenants (2 horas)

Crear la migración SQL con la tabla tenants más un tenant por defecto (el actual). Esto NO rompe nada existente porque el tenant_id tendrá un default temporal mientras migramos.

```
database/migrations/009_multitenant.sql
CREATE TABLE tenants (id UUID PRIMARY KEY, ...);
INSERT INTO tenants (id, nombre, subdomain) VALUES
  ('default', 'Consultorio Demo', 'demo');
ALTER TABLE pacientes ADD COLUMN tenant_id UUID REFERENCES tenants(id)
  DEFAULT 'default';
```

Paso 2: Extraer tenant del host en middleware (1 hora)

Modificar middleware.ts para detectar subdominio y setear x-tenant-id. Por ahora devuelve "default" hasta que tengamos más tenants.

Paso 3: PWA — manifest.json + service worker (1 hora)

Esto convierte el dashboard en una app instalable. El usuario puede ponerla en el escritorio de su celular/PC como si fuera una app nativa. Cero cambios en el backend.

```
app/manifest.ts  (dinámico, con nombre del tenant)
public/sw.js     (service worker básico con cache de assets)
layout.tsx       !' <link rel="manifest" href="/manifest" />
                  !' <meta name="theme-color" content={tenantColor} />
                  !' next.config.js: headers para SW
```

Paso 4: White-label desde .env.local (2 horas)

Mientras no tengamos la tabla tenants operativa, podemos leer LOGO_URL, COLOR_PRIMARY, NOMBRE_CONSULTORIO desde variables de entorno. Esto permite personalizar la instancia para cada cliente al deployar.

```
.env.example:
NEXT_PUBLIC_TENANT_NAME=Consultorio Demo
NEXT_PUBLIC_TENANT_LOGO=/logo.png
NEXT_PUBLIC_TENANT_PRIMARY=#2563eb

app/layout.tsx:
<link rel="icon" href={process.env.NEXT_PUBLIC_TENANT_LOGO} />
<meta name="theme-color" content={process.env.NEXT_PUBLIC_TENANT_PRIMARY} />

tailwind.config.ts:
primary: process.env.NEXT_PUBLIC_TENANT_PRIMARY || '#2563eb'
```

Paso 5: Landing page básica (3 horas)

Una página pública en / (cuando no hay sesión activa) que muestre el producto, features y pricing. Con un formulario de registro que cree un tenant nuevo. Podemos empezar con algo simple en app/page.tsx (actualmente es el login).

7. Beneficios Clave del Producto

- IA local sin costos recurrentes: Ollama + Mistral en tu VPS. Sin depender de OpenAI ni pagar por token.
- WhatsApp integrado: Los pacientes reciben recordatorios, confirmaciones y pueden sacar turnos desde WhatsApp. Sin instalar apps.
- Automatización completa: n8n gestiona recordatorios, resúmenes diarios, backup encriptado y triaje de mensajes.
- Dashboard completo desde el día 1: Turnos, pacientes, recetas, conversaciones, reportes, exportación Excel/PDF.
- Sin límite de pacientes en planes superiores: No hay cobro por paciente, solo costo fijo mensual.
- On-premise opcional: Clientes con requisitos de compliance pueden tener el sistema en su propia infraestructura.
- Multi-consultorio: Un médico con varias sucursales o múltiples profesionales en un mismo plan.
- Seguridad enterprise: 2FA, rate limiting, auditoría de accesos, bloqueo por intentos, backup encriptado.

Documento generado el 17/5/2026
AiCoreMed — Aicorebots.com